

Reporte de Agentic Loop

0. SKILLS CONFIGURADOS

arquitecto !' DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !' MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !' FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !' test (GLX-QA-VALIDACION-V1)
desarrollador_backend !' DESARROLLO_BACKEND-API (GLX-DESARROLLO_BACKEND-API-V1)

1. ANÁLISIS DE ARQUITECTURA

Micro-ADR: XP-14 — Página Web Metodologías TI

1. CONTEXTO

Se requiere una página web informativa/editorial sobre metodologías de informática y TI con 3 secciones de contenido (Home, Historias, Noticias), FAQ, aviso de cookies, analítica básica y medidas de seguridad XSS. El stack está explícitamente restringido a HTML5, CSS y JS puro. No hay backend, no hay autenticación, no hay base de datos. El entorno de ejecución final es un navegador via protocolo HTTP/HTTPS (servidor estático), pero dado que el requerimiento no especifica servidor Node/Python ni bundler, se diseña para máxima compatibilidad incluyendo apertura local (file:///).

2. DECISIÓN ARQUITECTÓNICA

****Monolito de Archivo Único con JS Vanilla empaquetado**** — Single HTML con CSS externo y JS externo plano (sin módulos ES6, sin import/export, sin bundler). Todo el JavaScript se consolida en un único archivo `main.js` con patrón IIFE (Immediately Invoked Function Expression) para encapsulamiento seguro. El CSS se separa en un único `styles.css`. Se utiliza una arquitectura de Single Page Application simulada mediante secciones visibles/ocultas con JS, sin routing real.

3. JUSTIFICACIÓN BASADA EN YAGNI / KISS

- ****¿Por qué NO React/Vue/Angular?*** YAGNI: No hay estado complejo, no hay llamadas a API dinámica, no hay componentes reutilizables entre proyectos. Añadir un framework SPA multiplica la complejidad de build, dependencias y tiempo de carga sin beneficio medible para contenido estático editorial.
- ****¿Por qué NO múltiples archivos JS con type='module'?*** El criterio de entorno obliga a garantizar compatibilidad file:/// y máxima compatibilidad de navegadores (Chrome, Firefox, Safari según CA-2). Los módulos ES6 nativos fallan en file:/// y requieren CORS headers en servidor. Un único main.js plano elimina ese riesgo.
- ****¿Por qué NO un SSG (Hugo, Jekyll, Eleventy)?*** El requerimiento especifica HTML5/CSS/JS directamente. Introducir un generador añade una capa de tooling que el equipo de QA y Dev debe mantener sin justificación en los criterios de aceptación.
- ****¿Por qué NO Google Tag Manager + backend para analytics?*** CA-5 pide Google Analytics. Se implementa via snippet estándar de GA4 inyectado directamente en el HTML, que es la solución canónica, madura y sin dependencias adicionales.
- ****¿Por qué NO DOMPurify via npm?*** Se carga DOMPurify desde CDN con hash de integridad (SRI) en el HTML. Esto satisface el criterio de seguridad XSS sin introducir un paso de build/bundling.
- ****Seguridad XSS****: Se implementa sanitización con DOMPurify para cualquier input de usuario (búsqueda/comentarios si aplica), escape de caracteres especiales via función utilitaria en main.js, y meta CSP en el HTML como capa de defensa en profundidad. HTTPS es responsabilidad del servidor de hosting (fuera del scope del código estático).

4. RIESGOS ASUMIDOS POR SIMPLIFICAR

- ****Riesgo 1 (BAJO)****: Sin bundler, el cache-busting de main.js y styles.css debe hacerse manualmente con query params (?v=1.0.0) al actualizar. Mitigación: versionado en el nombre del archivo o query string.
- ****Riesgo 2 (BAJO)****: Google Analytics requiere que la página esté servida via HTTP/HTTPS para funcionar. En entorno file:/// no reportará datos. Esto es esperado y documentado.
- ****Riesgo 3 (MEDIO)****: El Content-Security-Policy definido en meta tag tiene limitaciones vs header HTTP. Para producción, el servidor de hosting (Nginx, Apache, Netlify, Vercel) debe configurar el header CSP real. Se

documenta en README.

- **Riesgo 4 (BAJO)**: Todo el contenido editorial (textos de Home, Historias, Noticias, FAQ) se hardcodea en el HTML/JS como datos estáticos. Si el contenido cambia frecuentemente, se necesitará un CMS headless en el futuro. Aceptado para MVP.

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 3
- assets/js/main.js | Estado: APROBADO | Iteraciones: 2
- assets/css/styles.css | Estado: APROBADO | Iteraciones: 1
- README.md | Estado: SKIPPED (doc) | Iteraciones: 0

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 80,312

Tokens de salida (output): 48,566

Total tokens: 128,878

Modelo utilizado: claude-sonnet-4-6

Ø=Üh Ø=Ü» ~19.2 horas-hombre ahorradas

"H squad de 5 personas × 2.4 días

Desglose por Agente:

DISEÑO-ARQUITECTURA: !"910 input | !"1,455 output | 1 llamadas | ~1.5h ahorradas

MICROPLANIFICACION-DISEÑO: !"12,372 input | !"9,500 output | 4 llamadas | ~5.7h ahorradas

FRONTEND-DESARROLLO: !"38,800 input | !"27,319 output | 6 llamadas | ~6.5h ahorradas

test: !"28,230 input | !"10,292 output | 7 llamadas | ~5.5h ahorradas